

Canteen Ordering System

Documentation Set: Architecture · Design · User Guide

Architecture Document

1. Overview

The system is a client-server web application: a React single-page frontend talks to a Spring Boot REST API backend over HTTP/JSON, backed by an embedded H2 database. It supports two user experiences on the frontend: a Student page (browse menu, order, view own orders, rate completed orders) and an Admin page (manage menu items and stock, view all orders).

2. Components

- **Frontend (React)** — Student page and Admin page as separate views/routes; calls the backend REST API directly.
- **Controller layer** — AuthController, MenuController, OrderController, RatingController; translates HTTP requests to service calls and back to JSON responses.
- **Service layer** — OrderService and RatingService hold all business rules (stock, cutoff time, daily limits, cancellation window, rating eligibility); this is deliberately the only layer where these rules live.
- **Repository layer** — Spring Data JPA repositories (UserRepository, MenuItemRepository, OrderRepository, RatingRepository) handle persistence, with a couple of custom queries (e.g. summing a user's orders for a given day).
- **Security layer** — stateless JWT authentication: JwtAuthFilter reads the Authorization header on each request and sets the authenticated principal; SecurityConfig defines which routes are public vs protected.
- **Database** — H2, file-based for local development (data persists between restarts) and in-memory for the automated test suite (isolated, reset per run).

3. Data Flow — Placing an Order

Student page → POST /orders (JWT in Authorization header) → JwtAuthFilter resolves the User → OrderController → OrderService.placeOrder() checks cutoff time, stock, and daily limit in order → MenuItem stock is decremented via an optimistic-locking save → Order is persisted → OrderResponse JSON returned to the frontend.

If any check fails, OrderService throws an OrderException carrying the correct HTTP status (e.g. 409 for a business-rule conflict, 400 for bad input); GlobalExceptionHandler converts this into a consistent JSON error body instead of a raw stack trace.

4. Technology Choices

Choice	Reason
Spring Boot	Mature REST + JPA + Security stack; fast to build and test a real backend within a one-week time

H2 (embedded)	Zero external setup — a grader can run the project with no database installation, satisfying the 'sc
JWT (stateless auth)	No server-side session state to manage; scales cleanly and is a standard pattern for a REST API
Optimistic locking (@Version)	Prevents two concurrent orders from both succeeding against the same last unit of stock, without
React	Component-based frontend suited to two distinct views (Student/Admin) sharing one API.
JUnit 5 + Mockito + MockMvc	Unit tests isolate business rules from the database; MockMvc integration tests exercise the real H

5. Known Gap to Verify Before Defense

The core User entity, as originally designed, has no role field (student vs admin). If the admin-only endpoints (POST/PUT /menu) were added without a role check, any authenticated user — not just admins — could currently call them. Confirm whether role-based authorization was added for these endpoints; if not, this is worth fixing or at minimum disclosing as a known limitation, since Security is a scored criterion in this assessment.

Design Document

1. Data Model

Entity	Key fields	Relationships
User	id, username, passwordHash, displayName	One-to-many with Order, Rating
MenuItem	id, name, price, stockCount, dailyLimitPerPerson, version	One-to-many with Order
Order	id, quantity, totalPrice, status, createdAt	Many-to-one User, many-to-one MenuItem; one-to-one Rating
Rating	id, stars (1-5), review, createdAt	One-to-one Order (unique constraint: one rating per order), many-to-one User

2. Key Flows

Place order:

- Validate quantity ≥ 1 .
- Reject if current time is after the configured cutoff.
- Reject if requested quantity exceeds current stock.
- Reject if this would exceed the user's daily per-item limit.
- Decrement stock (optimistic-locked save) and persist the order.

Cancel order:

- Reject if the requester does not own the order.
- Reject if already cancelled.
- Reject if outside the cancellation grace window.
- Otherwise mark CANCELLED and restore the stock it had consumed.

Rate order:

- Reject if the requester does not own the order.
- Reject if the order is cancelled.
- Reject if the order already has a rating.
- Validate stars is between 1 and 5; persist the rating.

3. API Design

Method	Endpoint	Auth	Purpose
POST	/auth/register	No	Create account, returns JWT
POST	/auth/login	No	Authenticate, returns JWT
GET	/menu	No	List menu items with live stock
POST	/orders	Yes	Place an order
DELETE	/orders/{id}	Yes	Cancel own order within grace window

GET	/orders/mine	Yes	List own order history
POST	/ratings	Yes	Rate a completed order
GET	/ratings/menu-item/{id}	No	List ratings for a menu item
POST / PUT	/menu	Yes (admin)	Add or update a menu item / stock

4. Error Handling

All business-rule violations are raised as a single `OrderException` carrying an explicit `HttpStatus`, via named factory methods (e.g. `OrderException.outOfStock()`, `.cutoffPassed()`, `.alreadyRated()`) rather than scattering new-exception calls through the codebase. A single `GlobalExceptionHandler` converts these — plus framework-level exceptions such as validation failures, missing routes, and wrong HTTP methods — into a consistent JSON error shape (timestamp, status, error, message), and generic/unexpected exceptions are caught last and returned as a plain 500 with no internal details or stack trace exposed to the client.

User Guide

For Students

- **Sign up or log in** using your username and password.
- **Browse the menu** to see today's items, prices, and how many are left in stock.
- **Place an order** by choosing an item and quantity. Orders can only be placed before the daily cutoff time, and only up to your daily limit per item.
- **View your orders** to see everything you've ordered today, including their status.
- **Cancel an order** if you change your mind — but only within a few minutes of placing it. After that window, the order is locked in.
- **Rate an order** once it's complete: give it 1–5 stars and, if you like, a short written review. You can only rate an order once, and only if it wasn't cancelled.

For Canteen Admins

- **Log in** with an admin account.
- **Add a new menu item** — set its name, price, how many units are available today, and the maximum any one student can order.
- **Edit an existing item** — for example, top up stock partway through the day, or adjust the price.
- **View all orders** placed by students, to keep track of what's been ordered.

Good to Know

If an order is rejected, the reason is always shown plainly — for example “Ordering has closed for today” or “This item is out of stock” — so it's always clear why an action didn't go through, rather than a generic error.